dexamine: A Python package for Uniswap event data on Ethereum

Magnus Hansson[a,b,*]

[a]*Stockholm University, Stockholm, Sweden*
[b]*Swedish House of Finance, Stockholm, Sweden*

Abstract

Decentralized exchanges record trading and liquidity provision on public blockchains, but empirical analysis requires interpreting these records and linking them to execution metadata. dexamine is a Python package that parses Uniswap v2 and v3 events on Ethereum. It converts transaction receipt logs into observations of trades and liquidity changes, with token quantities, pool state, transaction order, and gas information. The package separates data retrieval, contract metadata, protocol interpretation, and output construction. A recorded transaction illustrates reproducible parsing without a live node. Version 1 has supported data construction for two studies of price discovery and price impact in decentralized markets.

Keywords: Ethereum, Uniswap, Decentralized finance, Market microstructure, Research software

*Corresponding author.
Email address: hansson.carl.magnus@gmail.com (Magnus Hansson)
URL: https://orcid.org/0009-0004-4318-5145 (Magnus Hansson)

Required Metadata

Nr.	Code metadata description	Metadata
C1	Current code version	1.1.0 (commit snapshot)
C2	Permanent GitHub link to code/repository used for this code version	`https://github.com/HanssonMagnus/dexamine/tree/d62095ab0544618f134cc25684b1203ffb476e53`
C3	Legal Code License	GNU General Public License v3.0 or later
C4	Code versioning system used	Git
C5	Software code languages, tools, and services used	Python; Ethereum JSON-RPC
C6	Compilation requirements, operating environments & dependencies	Verified in continuous integration on Linux with Python 3.10, 3.11, 3.12, and 3.13. Requires Python 3.10 or newer, web3 6.15.0 or newer, and requests 2.31.0 or newer. An Ethereum endpoint retaining the requested blocks and receipts is needed for live parsing. No compilation of dexamine is required.
C7	If available Link to developer documentation/manual	`https://github.com/HanssonMagnus/dexamine/blob/d62095ab0544618f134cc25684b1203ffb476e53/docs/docs.md`
C8	Support email for questions	`hansson.carl.magnus@gmail.com`

Table 1: Code metadata.

1. Motivation and significance

Decentralized exchanges allow users to trade digital assets through programs executed on a blockchain. Uniswap v2 and v3 are automated market makers on Ethereum: trades change the balances of liquidity pools, and prices follow rules implemented by their contracts [1, 2]. Their public records provide data for empirical studies of trading and liquidity provision. However, these records describe contract execution and must be interpreted before they become economic observations.

Studies of decentralized exchange market quality examine liquidity and trading costs as well as prices [3, 4]. Research on arbitrage and transaction ordering also shows why execution sequence and fees matter [5]. A transaction's position within a block, execution fees, and the pool state following a trade help describe the conditions under which it executed. One transaction can contain several trades or liquidity events, each of which must remain distinguishable. A block timestamp alone does not recover their execution order.

Ethereum exposes blocks, transactions, and event logs through JSON-RPC. Constructing a dataset requires protocol-specific decoding, resolution of token metadata, conversion of integer quantities into token units, and joins between events and execution metadata. Repeating these transformations across projects creates opportunities for inconsistent units, signs, and ordering. dexamine implements them in a reusable Python library. The researcher supplies transaction positions, defined by block number and transaction index, and receives records for the supported events.

Related tools address extraction and indexing at different levels. Ethereum ETL exports blockchain records and selected token data [6]. cryo supports bulk extraction into tabular formats and event decoding from supplied signatures [7]. Graph Node provides application-specific indexing and GraphQL queries through subgraphs and can be operated locally [8]. Its retained information depends on the chosen schema and mappings. TrueBlocks provides address-based transaction indexing [9]. These tools are discussed as related software; dexamine does not import or invoke them. Researchers can use external tools to select transactions before passing their positions to dexamine.

The contribution of dexamine is a common representation of Uniswap events joined to their execution metadata. Generic extraction tools still require these transformations, while a subgraph places them within an indexing service. A separate library allows researchers to reuse RPC and contract-access facilities while incorporating the protocol interpretation into their own pipelines. Transaction discovery, storage, and statistical estimation remain independent stages. Version 1 has been used for data construction in two studies of decentralized markets [10, 11].

2. Software description

2.1. Software architecture

Figure 1 shows the separation of JSON-RPC retrieval, contract metadata resolution, protocol parsers, and output construction. A reusable session loads contract interfaces and retains pool and token metadata across calls. dexamine's JSON-RPC client uses Requests [12] to retrieve transactions, receipts, and blocks over HTTP. web3.py [13] supplies contract calls for pool and token metadata and address checksum conversion. Requests and web3.py are the two declared runtime dependencies. The protocol parsers decode event topics and fixed-width data fields directly in Python. The output layer joins these event records to execution metadata.

Requests for multiple positions are batched. Results are yielded incrementally, so event records need not accumulate in memory. The metadata cache grows with the number of distinct contracts encountered. Users can divide positions across processes with one session per worker. Processing rates depend on endpoint latency and capacity, batch size, and metadata cache misses; no throughput advantage over other extraction tools is claimed here.

2.2. Software functionalities

The package supports Uniswap v2 and v3 swaps, mints, and burns on Ethereum mainnet. Mints and burns describe additions and removals of liquidity. Event records include token symbols and decimal precision, normalized token quantities, and event-specific pool information. The sign conventions express liquidity additions as positive quantities and removals as negative quantities; swap quantities describe the pool's net token flows.

The parsers account for protocol differences. Uniswap v2 reports reserve updates in a preceding `Sync` event. dexamine checks that the immediately preceding log is a `Sync` from the same pool and skips an event if this condition fails. For swaps, the reserve ratio supplies the pool's post-event marginal price before fees. Uniswap v3 swap logs report price, tick, and active liquidity directly. The parser uses these values to derive price and virtual reserves. These reserves describe the local trading curve, rather than total pool balances; they are reported as missing when active liquidity is zero.

The output can retain node payloads alongside parsed events or provide one flat row per event. Flat records include block and transaction indices, the transaction hash, and the event's zero-based position within the receipt's log list. This last field is named `receipt_log_index`; it differs from Ethereum's block-wide `logIndex`. Together, transaction identity and receipt position identify the source event. Missing or inapplicable numerical fields are represented by `None`, which becomes `null` in JSON.

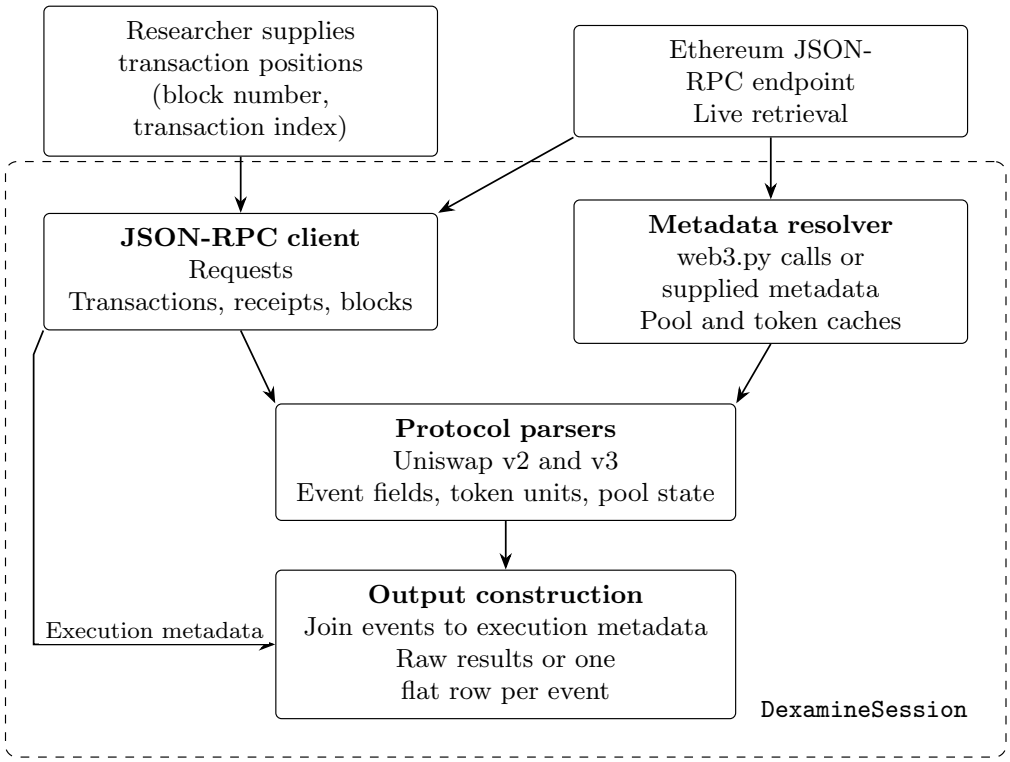

Figure 1: Data flow within a session. The RPC client retrieves execution records; the metadata resolver supplies pool and token information. Protocol parsers decode receipt logs, and the output layer joins the resulting events to transaction and block metadata. Offline replay supplies recorded RPC results and seeded metadata.

Token amounts are scaled by their decimal precision. Normalized quantities use floating-point arithmetic and can contain rounding error, so the output is intended for empirical analysis rather than exact integer accounting. Retaining the original logs allows users to recover integer quantities. Gas consumption and fee fields apply to the entire transaction and are repeated across its events. They do not allocate execution costs to individual trades.
Destination labels compare the transaction's top-level destination against a bundled list of Uniswap router and periphery addresses (Figure 2). Other destinations and contract creation receive separate labels. This classification does not reconstruct internal call paths or establish trader identity, arbitrage, or maximal extractable value. Those interpretations require additional evidence.

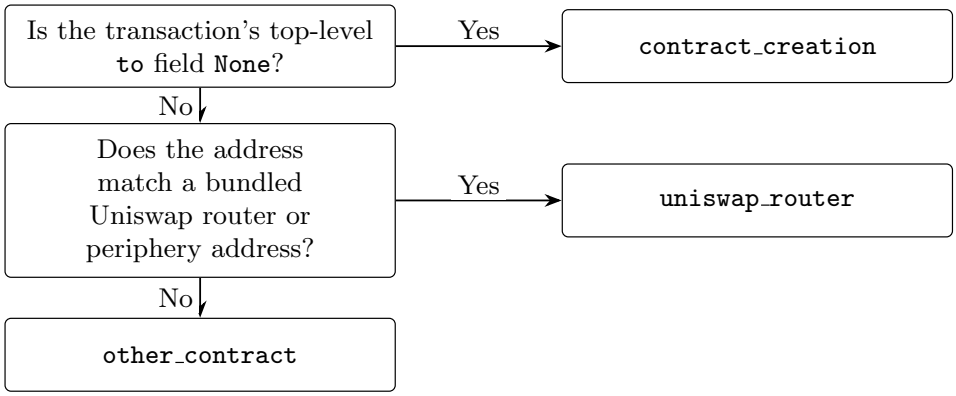

Figure 2: Construction of the `tx_to_type` label from the transaction's top-level destination. The `other_contract` label covers every remaining destination; it does not establish an address's economic role or reconstruct the path to a pool.

2.3. Reproducibility and verification

The author wrote the original implementation without generative AI assistance; the repository's first commit is dated 25 November 2023. In subsequent development, the author used Anthropic Claude and OpenAI ChatGPT to assist with writing tests, reviewing code, and updating the repository. The commit history documents the development sequence and includes explicit AI attributions for later changes. For this article, OpenAI Codex (GPT-6) also assisted with the metadata-seeding API and its tests.
Reproducibility depends on the software version, source responses, and resolved metadata. Pool and token metadata are queried at the latest block and cached. Changes to token metadata can therefore affect repeated runs even when historical logs are unchanged. Retaining input responses and metadata supports replication. The public `MetadataResolver.seed` method accepts

recorded token and pool metadata, normalizes addresses, and replaces the supplied cache entries. Missing entries still trigger endpoint queries; callers retain responsibility for the provenance of supplied metadata. The endpoint must serve the requested historical blocks and receipts and support metadata calls, but the parser does not require historical contract-state queries. Users should also retain transaction hashes when selecting positions, since chain reorganizations can change the transaction at a recent position.
Offline tests exercise event decoding, signed quantities, reserve ordering, pool filtering, and missing values using recorded responses and constructed event logs. An optional integration suite checks the public interface against a live endpoint. Continuous integration runs tests, formatting, linting, and strict type checking on Python 3.10 through 3.13. Installation instructions, field definitions, limitations, and contribution guidance are supplied with the code.

3. Illustrative examples

Consider transaction 59 in Ethereum block 12,376,729, which created the Uniswap v3 USDC/WETH pool with a 0.05% fee and minted its first liquidity position. The following example retrieves its parsed event from an endpoint that retains the required history:

```
from dexamine import DexamineSession

session = DexamineSession.from_node_url(node_url)
rows = session.parse_position(
    block_number=12376729,
    tx_index=59,
    protocol="uniswap_v3",
    exchange_pair_address=
        "0x88e6A0c2dDD26FEEb64F039a2c41296FcB3f5640",
    output_format="flat",
)
```

The result contains one mint event. Table 2 gives selected fields; token quantities are rounded for display. The mint is the sixth log in the receipt, so its receipt position is 5. The USDC quantity is divided by 10^6 and the WETH quantity by 10^{18}. Their positive signs indicate liquidity supplied to the pool. The tick bounds identify the price range of that position. Price and virtual reserves are absent because this is a mint record, not a swap record. The gas usage includes pool creation and other operations in the transaction and must not be interpreted as the cost of minting alone. Similarly, the destination label reflects the Uniswap position-manager address; it does not identify the transaction's economic purpose. Joining such records across trans-

Field	Value
Event type	mint
Token pair	USDC / WETH
Receipt log position	5
USDC amount	2995.507735
WETH amount	0.999999999872
Lower / upper tick	191150 / 198080
Transaction gas used	5,201,405
Effective gas price	64 gwei
Destination label	`uniswap_router`

Table 2: Selected output from the recorded Uniswap v3 transaction. Token amounts are rounded; gas usage refers to the whole transaction.

actions yields ordered observations of liquidity provision without assigning those interpretations inside the parser.

The accompanying script `parse_recorded_transaction.py` reproduces the complete output offline using the repository's recorded transaction, receipt, and block responses, with explicitly supplied USDC, WETH, and pool metadata. It replays the public RPC client's `call` method and uses `MetadataResolver.seed` while exercising the same session, parser, and output construction. Unexpected HTTP access is rejected. The committed JSON output can be checked with `--check`; `--node-url` instead enables live retrieval. The example demonstrates the transformation of one historical transaction, rather than reproducing either complete research dataset.

4. Impact

Version 1 of dexamine was used to construct data for *Price Discovery in Constant Product Markets* [10] and *Measuring DeFi Price Impact and A New Empirical Market Microstructure Model* [11]. The former studies the contribution of trading and liquidity provision to price discovery on Uniswap. The latter develops an empirical market microstructure model for decentralized exchanges that distinguishes arbitrageurs when analyzing returns and order flow. These applications use the parser as a data-construction component; trader classification and econometric estimation are performed separately.

Both applications require preserving the sequence of events and their associated pool information. Reusing a parser provides consistent treatment of token units, liquidity changes, and execution metadata across analyses. It also makes the data transformations inspectable independently of the statistical model. The research use of version 1 predates the first public release. The

code metadata identifies the 1.1.0 snapshot described here, which adds the public metadata-seeding interface used by the offline example.
The same representation supports further studies of liquidity provision, execution costs, and price adjustment. Researchers can select different transaction samples or apply alternative classifications while retaining the event construction. For example, a study of gas costs can aggregate at transaction level before linking costs to a set of events, while a study of liquidity can retain separate mint and burn observations. The available evidence of use is the two research applications cited above. The paper does not report adoption counts or commercial deployments.

5. Conclusions

dexamine converts Uniswap v2 and v3 receipt logs into event observations linked to Ethereum execution metadata. Its contribution is a reusable implementation of protocol interpretation and data joins for empirical research. The library supports both individual transactions and batched processing, with offline verification and an executable example. Its scope and numerical conventions are explicit: transaction discovery, trader attribution, and statistical analysis remain the responsibility of the surrounding research workflow.

Acknowledgements

I thank the TrueBlocks [9] and Erigon [14] teams for assistance with node operation and indexing, and members of the Flashbots and Uniswap communities for discussions of the protocols.

Funding

During preparation of this manuscript, the author received support from Handelsbanken Research Foundations through a Browaldh Scholarship.

CRediT authorship contribution statement

Magnus Hansson: Conceptualization, Methodology, Software, Validation, Writing (original draft), Writing (review and editing).

Data availability

The software is publicly available from the GitHub repository identified in the code metadata table. Recorded Ethereum responses used in the illustrative example are included in the repository under `dexamine/tests/test_data/node_responses`. The example script and expected output accompany this manuscript. The complete datasets of the cited research studies are not distributed with this example.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

The original software was written by the author without generative AI assistance. Later use of Claude and ChatGPT for software maintenance is described in Section 2.3. OpenAI Codex (GPT-6) assisted with manuscript organization and editing, reference checking, revisions to the TikZ figures, the reproducible example, and the publication build. The author reviewed manuscript drafts and supplied corrections and clarifications. Checks performed during revision included comparison with source code and cited sources, offline execution of the example, and manuscript compilation. The author remains responsible for the manuscript and software.

References

[1] H. Adams, N. Zinsmeister, D. Robinson, Uniswap v2 core, Tech. rep., Uniswap (2020).
URL `https://uniswap.org/whitepaper.pdf`

[2] H. Adams, N. Zinsmeister, M. Salem, R. Keefer, D. Robinson, Uniswap v3 core, Tech. rep., Uniswap (2021).
URL `https://uniswap.org/whitepaper-v3.pdf`

[3] A. Lehar, C. Parlour, Decentralized exchange: The Uniswap automated market maker, The Journal of Finance 80 (1) (2025) 321–374. `doi:10.1111/jofi.13405`.

[4] A. Barbon, A. Ranaldo, On the quality of cryptocurrency markets: Centralized vs. decentralized exchanges, Management Science (2026). `doi:10.1287/mnsc.2024.07703`.

[5] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, A. Juels, Flash Boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability, in: 2020 IEEE Symposium

on Security and Privacy (SP), 2020, pp. 910–927. doi:10.1109/SP40000.2020.00040.

[6] Ethereum ETL contributors, Ethereum ETL: Export blockchain data to CSV or JSON files, accessed 8 September 2026 (n.d.).
URL https://github.com/blockchain-etl/ethereum-etl

[7] Paradigm, cryo: Extract blockchain data to Parquet, CSV, JSON or Python dataframes, accessed 8 September 2026 (n.d.).
URL https://github.com/paradigmxyz/cryo

[8] The Graph Protocol, Graph Node: An indexing and query layer for blockchain data, accessed 8 September 2026 (n.d.).
URL https://github.com/graphprotocol/graph-node

[9] TrueBlocks Team, TrueBlocks / Unchained Index, accessed 8 September 2026 (n.d.).
URL https://github.com/TrueBlocks/trueblocks-core

[10] M. Hansson, Price discovery in constant product markets, SSRN working paper (2024). doi:10.2139/ssrn.4582649.

[11] M. Hansson, E. Ghysels, G. Nguyen, X. Li, Measuring DeFi price impact and a new empirical market microstructure model, SSRN working paper (2025). doi:10.2139/ssrn.5261306.

[12] K. Reitz, Requests contributors, Requests: HTTP for humans, accessed 8 September 2026 (n.d.).
URL https://requests.readthedocs.io/en/latest/

[13] web3.py contributors, web3.py: A Python library for interacting with Ethereum, accessed 8 September 2026 (n.d.).
URL https://web3py.readthedocs.io/en/stable/

[14] Erigon contributors, Erigon: An Ethereum implementation on the efficiency frontier, accessed 8 September 2026 (n.d.).
URL https://github.com/erigontech/erigon